

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a49e389804868334d.jsonl`  
- SHA-256: `62641ba2220f3bf1628b49fc42e255012c4342564406a8ef2cc3e9e3d058b8c9`  
- Source modified: `2026-06-22T15:05:32+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_f39953fd-98d`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:03:09.422Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is `real=false` if it doesn't hold up.

Lens: robustness-security

Severity: major

Title: Missing/garbled intervals.csv after a `rc=0` marker raises an uncaught exception instead of a clean failure

File: backend/main.py:518-520

Detail: After `result.returncode == 0`, the code unconditionally calls `_ingest_remote_segments(video_id, result.outputs_uri, ...)`. That function does `storage.fetch(uri, dst, ...)` with no error handling. If the worker wrote the done-marker (with a non-empty `video_id` and `returncode=0`) and report.json but intervals.csv is absent or unreadable in the bucket (partial write, race, GCS eventual-consistency, or an empty-rally run that simply didn't emit the CSV), `GCSStorage.fetch_to_local` calls `download_to_filename` which raises `NotFound`. That exception escapes `_ingest_remote_segments` and `_maybe_dispatch_remote` entirely (it is outside the `run_infer` try/except), bypassing the careful `_failed(...)` shape the function uses everywhere else, and becomes a raw 500-shaped internal job error. The whole cloud path otherwise goes to great lengths to convert every dispatch/poll error into a clean `_failed`; this one read can't.

Fix: Wrap the fetch+parse in `_ingest_remote_segments` (or its call site at main.py:520) in try/except and route a read/parse failure through `_failed('cloud-serving: could not read intervals.csv from <uri>: ...')`, consistent with the `run_infer` error branch. A missing-CSV regression test against a fake client that omits the CSV would lock this in.

2. assistant (2026-06-22T15:03:11.961Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repo.

3. user (2026-06-22T15:03:14.948Z)

```
backend/main.py | 100 +++++  
tests/test_api_cloud_dispatch.py | 212 +++++  
2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:03:17.171Z)

Let me look at the actual code in main.py around the cited lines and the relevant functions.

5. user (2026-06-22T15:03:18.386Z)

```
diff --git a/backend/main.py b/backend/main.py  
index 4e5a19a..04ffb67 100644
```

```

--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
import os
import sys
import argparse
+import csv
import json
import logging
import shutil
import socket
+import tempfile
import threading
import uuid
import webbrowser

@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
+    bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
+    produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
+    (cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
+    ``deployment.compute_target
+    == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
+    intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
+    ``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
+    ``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""

```

```

+ from backend.compute import ComputeJobSpec, get_compute_backend
+
+ compute = get_compute_backend(global_config)
+ if compute is None:
+     return None # default-OFF ? run the in-process segmenter below (unchanged)
+
+ storage_cfg = getattr(global_config, "storage", None)
+
+ def _failed(msg: str) -> Dict[str, Any]:
+     # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+     return {"video_id": video_id, "status": "failed", "message": msg,
+           "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+ # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+     try:
+         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+     except ValueError as e:
+         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+     if input_ref.backend == "local":
+         return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+
+                         "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+     if not out_root:
+         return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+
+                         "? that's where the Cloud Run job writes the result.")
+
+     notify("dispatching (cloud_run)")
+     logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+     spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+     try:
+         result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+         return _failed(f"cloud dispatch error: {e}")
+     if result.returncode != 0:
+         return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+     notify("ingesting (cloud_run)")
+     storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+     _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+     segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+     _finalize_reingest(video_id, segments, play_wm, cand_wm)
+     return {"video_id": video_id, "status": "success",
+           "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+           "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
+     """The full hash ? validate ? segment ? report pipeline for one video.
@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:

```

```

        # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
        # if the run yields segments; otherwise drop the new partial rows and keep the old.
        play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+       # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+       # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+       # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+       # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+       # they're dropped on the cloud path in this slice ? a documented follow-up.)
+       remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+       if remote is not None:
+           segmentation_ran = True
+           response = remote
+           return response
        segmenter = segmenter_cls(db_instance, global_config)
        try:
            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)

```

6. assistant (2026-06-22T15:03:21.041Z)

Now let me look at the broader context of `\_execute\_processing` to confirm the try/except structure and whether `\_maybe\_dispatch\_remote` (and thus `\_ingest\_remote\_segments`) is inside a protective try/except.

7. user (2026-06-22T15:03:22.082Z)

```

528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]

```

```

557
558     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560     seg_name = req.segmenter or default_seg
561     segmenter_actual = None
562     segmentation_ran = False
563     response: Optional[Dict[str, Any]] = None
564     try:
565         if failures:
566             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567             # status; it no longer destroys the video's prior good segments.
568             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569             response = {
570                 "video_id": video_id,
571                 "status": "failed",
572                 "message": "Video failed validation checks.",
573                 "results": [r.model_dump() for r in val_results],
574             }
575             return response
576
577     # 2. Run segmenter & indexer
578     logger.info("[API] Video passed checks. Segmenting and indexing...")
579     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581     segmenter_cls = segmenter_registry.get(seg_name)
582     if not segmenter_cls:
583         # Submit-time validation already 400s unknown names; this is the defensive
584         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585         available = list(segmenter_registry.list_available().keys())
586         response = {
587             "video_id": video_id,
588             "status": "failed",
589             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590             "results": [r.model_dump() for r in val_results],
591             "play_segments": [],
592         }
593         return response
594
595     logger.info(f"[API] Using segmenter: {seg_name}")
596     notify(f"segmenting ({seg_name})")
597     segmenter_actual = seg_name
598     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605     # they're dropped on the cloud path in this slice ? a documented follow-up.)
606     remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607     if remote is not None:
608         segmentation_ran = True
609         response = remote

```

```

610         return response
611     segmenter = segmenter_cls(db_instance, global_config)
612     try:
613         result = segmenter.process_video(video_id, local_video, req.player_pool,
614 req.max_frames)
615     except Exception:
616         # A raising segmenter must not leave half-written rows: restore the pre-run
617         # state (same destroy-after-confirm contract as the Failure branch), then
618         let
619         # the job worker record the failure.
620         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
621         raise
622     segmentation_ran = True
623
624     if isinstance(result, Failure):
625         logger.error(f"[API] Segmenter failed: {result.message}")
626         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
627         response = {
628             "video_id": video_id,
629             "status": "failed",
630             "message": f"Segmenter '{seg_name}' failed: {result.message}",
631             "results": [r.model_dump() for r in val_results],
632             "play_segments": [],
633         }
634         return response
635
636     segments = result.value if hasattr(result, "value") else result
637     notify("finalizing")
638     _finalize_reingest(video_id, segments, play_wm, cand_wm)
639
640     response = {
641         "video_id": video_id,
642         "status": "success",
643         "message": f"Successfully indexed {len(segments)} play segments using
644 '{seg_name}' segmenter.",
645         "results": [r.model_dump() for r in val_results],
646         "play_segments": segments,
647     }
648     return response
649 finally:
650     # Always emit the per-video observability report (next to the video, or to

```

8. assistant (2026-06-22T15:03:26.998Z)

Now I have the key structural facts. The finding's claim is that the exception "escapes... and becomes a raw 500-shaped internal job error" and that it's "outside the run_infer try/except". Let me verify two important things:

1. Whether `\_maybe\_dispatch\_remote` (line 606) ? and thus the `\_ingest\_remote\_segments` call at line 590 of the diff ? is inside the outer `try`/`finally` at line 564.
2. What the finding claims about the file:line ? it cites `main.py:518-520`, but the actual ingest call is at line 590 of the new file. Let me check the exact line numbers.

9. user (2026-06-22T15:03:28.001Z)

```

441     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) ->
442 int:
443         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
444 the bucket and
445 persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
446 call ? so
447         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
448 were produced.
449         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
450 pattern (cloud

```

```

446         backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest.""
447         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
448         ref = storage.parse_uri(uri)
449         dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or
"intervals.csv")
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n
469
470
471     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
472                                play_wm: int, cand_wm: int, notify) -> Optional[Dict[str,
Any]]:
473         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
474         == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
475         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
476
477         Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
478         fall through to the in-process segmenter ? **DEFAULT-OFF*: ``get_compute_backend``
returns ``None``
479         for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
480         from backend.compute import ComputeJobSpec, get_compute_backend
481
482         compute = get_compute_backend(global_config)
483         if compute is None:
484             return None # default-OFF ? run the in-process segmenter below (unchanged)
485
486         storage_cfg = getattr(global_config, "storage", None)
487
488         def _failed(msg: str) -> Dict[str, Any]:
489             # Shape-match the in-process segmenter-fail branch + restore prior good rows
(none written yet).
490             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
491             return {"video_id": video_id, "status": "failed", "message": msg,
492                     "results": [r.model_dump() for r in val_results], "play_segments": []}
493
494         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
495         try:
496             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
497         except ValueError as e:
498             return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
499         if input_ref.backend == "local":
500             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to

```

```

this machine "
501                 "can't be read by the Cloud Run job. Put the video in your bucket
first.")
502         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
503         if not out_root:
504             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
505                             "? that's where the Cloud Run job writes the result.")
506
507         notify("dispatching (cloud_run)")
508         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
509         spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
510         try:
511             result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
512         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
513             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
514             return _failed(f"cloud dispatch error: {e}")
515         if result.returncode != 0:
516             return _failed(f"cloud job failed (returncode {result.returncode}).")
517
518         notify("ingesting (cloud_run)")
519         storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.

```

10. assistant (2026-06-22T15:03:34.296Z)

The cited line `main.py:518-520` exactly matches the `\_ingest\_remote\_segments` call site (line 520 calls it, after the `result.returncode != 0` check at 515-516, which is after `result.returncode == 0` implicitly). The finding's line citation is accurate.

Now the key question: does the exception "escape `\_maybe\_dispatch\_remote` and `\_execute\_processing` entirely... outside the run_infer try/except, bypassing the careful `\_failed(...)` shape"? Let me verify the structure of `\_finalize\_reingest` (also called at line 523, before `\_ingest\_remote\_segments`'s failure could surface ? actually after). And critically, I need to verify whether there's an outer try/except in `\_execute\_processing` that would catch it. We saw the `try:` at line 564 has a `finally:` at line 646 but I need to check if there's an `except` clause too.

11. user (2026-06-22T15:03:35.349Z)

```

646         finally:
647             # Always emit the per-video observability report (next to the video, or to
648             # report.output_dir). Never raises. Surface the paths in the response.
649             paths = emit_indexer_report(
650                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
651                 val_results=val_results, val_context=val_context,

```



```

652         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
653         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
654     )
655     if paths and isinstance(response, dict):
656         response["reports"] = paths
657
658
659     @app.post("/api/process", status_code=202)
660     def process_video(req: ProcessRequest, request: Request):
661         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
662
663         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
664         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
665         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
666         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
667         """
668         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
669         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
670         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
671         try:
672             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
673         except edge_auth.EdgeAuthError as e:
674             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
675
676         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
677         try:
678             validate_input_path(req.video_path, allowed_root=allowed_root)
679             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
680             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
681             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
682         except ValueError as e:
683             raise HTTPException(status_code=400, detail=str(e)) from e
684         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
685         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
686         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
687         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
688         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
689         # rejects a non-local URI as out-of-root.
690         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
691             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
692         if req.segmenter and not segmenter_registry.get(req.segmenter):
693             available = list(segmenter_registry.list_available().keys())
694             raise HTTPException(
695                 status_code=400,
696                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
697     )
698
699     job_id = uuid.uuid4().hex
700     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
701                                 req.player_pool, req.max_frames, owner_id=owner_id,
702                                 locale=req.locale):
703         raise HTTPException(status_code=500, detail="Failed to persist job record.")

```

```
704         _get_executor().submit(_drain_due_jobs)
705     return JSONResponse(
```

12. assistant (2026-06-22T15:03:40.168Z)

Confirmed: the outer block in `\_execute\_processing` is `try: ... finally:` ? there is NO `except` clause. So an exception raised inside `\_maybe\_dispatch\_remote` (called at line 606, inside the try) propagates up through the `finally` (which only emits a report and never raises) and out of `\_execute\_processing`. Let me confirm how the caller (`\_drain\_due\_jobs` / job worker) treats a raised exception ? does it become a "raw 500-shaped internal job error" as claimed, or is it handled cleanly?

13. user (2026-06-22T15:03:41.894Z)

```
237:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
245:    _drain_due_jobs,
516:    return _failed(f"cloud job failed (returncode {result.returncode}).")
529:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
704:    _get_executor().submit(_drain_due_jobs)
862:    # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
1069:    # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-released
on exit
1094:    logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")
1206:    _get_executor().submit(_drain_due_jobs)
```

14. assistant (2026-06-22T15:03:44.176Z)

`\_drain\_due\_jobs` is imported from another module. Let me find its definition to see how it handles an exception from `\_execute\_processing`.

15. user (2026-06-22T15:03:45.759Z)

(Bash completed with no output)

16. user (2026-06-22T15:03:49.269Z)

```
backend/api/job_worker.py
backend/api/__pycache__/job_worker.cpython-313.pyc
backend/main.py
backend/__pycache__/main.cpython-313.pyc
---
backend/main.py:529:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
backend/main.py:862:    # for a bucket URI (the same seam the process path uses,
_execute_processing main.py:159).
```

17. user (2026-06-22T15:03:52.592Z)

```
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
44:    NOT a failure. The worker persists `next_poll_at` + `progress` (via `set_job_waiting`),
46:    is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
76:    Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done' means
77:    the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78:    result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means the
86:    result = _main._execute_processing(
87:        req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
89:    db_instance.set_job_video_id(job_id, result["video_id"])
95:    db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96:    except Exception as e:
98:    db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
113:    Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job`` is
the
128:    except Exception as e: # never wedge a completed job on bookkeeping
```

```

129:         logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)
132: def _drain_due_jobs() -> int:
163:     except Exception as e: # pragma: no cover - defensive; never wedge the worker

```

18. assistant (2026-06-22T15:03:55.367Z)

This is the crux. Let me read the exact handler around lines 86-98 of job_worker.py.

19. user (2026-06-22T15:03:56.006Z)

```

70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(
87                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88             if result.get("video_id"):
89                 db_instance.set_job_video_id(job_id, result["video_id"])
90             db_instance.mark_job_done(job_id, result)
91             _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
92         except _main.JobWaiting as w:
93             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96         except Exception as e:
97             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105         persist it.
106
107         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
109         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111         With the placeholder pricebook the cost is 0 regardless.
112
113         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
114         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

```

20. assistant (2026-06-22T15:04:06.992Z)

This is the decisive evidence. The worker DOES catch any exception from `\_execute\_processing` at line 96-98: it logs the traceback and calls `db\_instance.mark\_job\_failed(job\_id, ...)`. So a `NotFound` raised by `\_ingest\_remote\_segments` does NOT become a "raw 500-shaped internal job error" ? it becomes a **clean `mark\_job\_failed`** job state, which is exactly the terminal-failed state the worker is designed to record.

Now let me assess the finding's accuracy point by point. The finding's central claim of severity (major) rests on the assertion that the exception "becomes a raw 500-shaped internal job error" and "bypasses the careful `\_failed(...)` shape". Let me verify what `mark\_job\_failed` produces vs what `\_failed` produces, to judge the real-world delta.

The key distinction:

- `\_failed(...)` path: returns a dict `{status: "failed", message: <specific>, ...}`, the worker calls `mark\_job\_done(job\_id, result)` ? job status `done`, result status `failed`. Plus it calls `prune\_segments` to restore prior good rows.
- Uncaught-raise path: worker catches it, calls `mark\_job\_failed(job\_id, "NotFound: ...)` ? job status `failed`. The report-emitting `finally` still runs.

So there ARE two real differences. Let me check whether the segment-restoration (prune) matters here ? i.e., whether rows could have been partially written before the CSV read fails. Looking at `\_ingest\_remote\_segments`: the `storage.fetch` (line 450) happens BEFORE any `add\_play\_segment`. So if `fetch` raises `NotFound`, zero rows were written ? there's nothing to prune. Let me also confirm the finding's specific failure scenario about an empty-rally run.

Let me verify the worker-side `infer` to see whether an empty-rally run actually omits `intervals.csv` (the finding's most plausible trigger), and what `storage.fetch`/GCSStorage does on a missing object.

21. user (2026-06-22T15:04:08.880Z)

```
backend/storage/__init__.py:106: def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None,
*,
backend/storage/__init__.py:111:     return get_storage(ref.backend,
config=config).fetch_to_local(ref, dst, overwrite=overwrite)
backend/storage/base.py:18:     """Backend-agnostic object storage. ``fetch_to_local`` + ``put``
are the two the
backend/storage/base.py:22:     def fetch_to_local(self, ref: StorageRef, dst: "str |
os.PathLike[str] | None" = None,
backend/storage/base.py:48:         ``NotImplementedError`` as expected and fall back to
``fetch_to_local``. """
backend/storage/gcs.py:30:     def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend/storage/gcs.py:32:         raise ValueError("GCSStorage.fetch_to_local requires a dst
path")
backend/storage/gcs.py:38:         self._blob(ref).download_to_filename(dst)
backend/storage/gdrive.py:101:         isn't synced ? raise with that remedy instead of a cryptic
FileNotFoundError downstream. """
backend/storage/gdrive.py:105:         raise FileNotFoundError(_not_synced_msg(self._root,
rel, ref.uri))
backend/storage/gdrive.py:112:     def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend/storage/gdrive.py:116:         return self._local.fetch_to_local(self._local_ref(ref),
dst, overwrite=overwrite)
backend/storage/local.py:3: Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an
identity passthrough (the
backend/storage/local.py:29:     def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend/storage/s3.py:38:     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend/storage/s3.py:40:         raise ValueError("S3Storage.fetch_to_local requires a dst
path")
backend/worker/__main__.py:73: def _write_intervals_csv(segments: List[dict], path: str) -> None:
backend/worker/__main__.py:241:     _write_intervals_csv(segments, os.path.join(out_local,
"intervals.csv"))
```

22. user (2026-06-22T15:04:11.727Z)

```
28         return self._client.bucket(ref.bucket).blob(ref.key)
29
30     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31         if dst is None:
32             raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33         import os
34         dst = str(dst)
35         if os.path.exists(dst) and not overwrite:
36             return dst
37         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38         self._blob(ref).download_to_filename(dst)
39         return dst
40
41     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43         return ref
44
45     def exists(self, ref) -> bool:
46         return bool(self._blob(ref).exists())
47
```

23. assistant (2026-06-22T15:04:15.780Z)

Confirmed: `download_to_filename` on a missing GCS blob raises `google.cloud.exceptions.NotFound` (an uncaught exception). Now let me check the worker side: does an empty-rally run still write `intervals.csv`? Line 241 unconditionally calls `_write_intervals_csv(segments, ...)` even for empty segments. Let me verify.

24. user (2026-06-22T15:04:16.406Z)

```
73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second `[start,end)` rallies + shot count / ending reason ? the join-
friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
78             w.writerow(["start", "end", "shot_count", "ending_reason"])
79             for s in segments:
80                 w.writerow([
81                     f'{float(s.get("start_time", 0.0)):.3f}',
82                     f'{float(s.get("end_time", 0.0)):.3f}',
83                     s.get("shot_count", ""),
84                     s.get("ending_reason", s.get("shot_count_confidence", "")),
85                 ])
86
87
88     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
```

25. assistant (2026-06-22T15:04:20.565Z)

So in the worker's normal flow, an empty-rally run **does** write `intervals.csv` (a header-only CSV) ? line 241 calls it unconditionally, and `csv.DictReader` over a header-only file yields zero rows (clean). So the finding's "empty-rally run that simply didn't emit the CSV" sub-scenario is **incorrect** for the happy path ? the worker always writes the CSV. Let me confirm line 241 is unconditional (not inside a `if segments:` guard).

26. user (2026-06-22T15:04:21.256Z)

```

230         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
231         emit_indexer_report(
232             db=db, video_id=video_id, video_path=local_video, config=config,
233             val_results=val_results, val_context=val_context,
234             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
235             was_reingest=False, segmentation_ran=segmentation_ran,
236         )
237
238         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
239         out_local = os.path.join(scratch, "outputs", video_id)
240         os.makedirs(out_local, exist_ok=True)
241         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
242         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
243
244         out_prefix = args.out_uri.rstrip("/")
245         pushed = []
246         for fn in sorted(os.listdir(out_local)):
247             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
248             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
249             pushed.append(uri)
250
251         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
252         for u in pushed:
253             print("    ", u)
254
255         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
256         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
257         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
258         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
259         if getattr(args, "done_uri", None):
260             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
261         return 0
262
263
264         #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a

```

27. assistant (2026-06-22T15:04:25.663Z)

Now I have a complete and precise picture. Let me also confirm the `result.outputs\_uri` value, to make sure the ingest is reading from the right place ? there could be a path-mismatch bug (a different real defect than the one claimed). Let me check what `run\_infer` sets `outputs\_uri` to and whether it matches `{out\_uri}/outputs/{video\_id}`.

28. user (2026-06-22T15:04:27.035Z)

```

7: 2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
11: 3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12:     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
106:     done_uri = f"{out_root}/_dispatch/{token}.done"
108:         "--done-uri", done_uri]
117:         self._job_name, execution, done_uri)
119:     marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
121:     video_id = str(marker.get("video_id", ""))
125:     if not video_id:
127:         "video_id" ? treating as failure", self._job_name)

```

```

129:         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130:         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131:         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc)
132:         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
135:     def _read_marker(self, done_uri: str) -> Optional[dict]:
137:         ref = parse_uri(done_uri)
140:         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)

```

29. assistant (2026-06-22T15:04:30.346Z)

This is important corroboration. Let me read the `run\_infer` flow around lines 119-132 to understand the marker/report read, and whether report.json read failure is already handled ? this tells me whether `\_read\_json` already provides a precedent for graceful handling that `\_ingest\_remote\_segments` lacks.

30. user (2026-06-22T15:04:31.162Z)

```

119         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                               label="cloud-run-infer", logger=LOG)
121         video_id = str(marker.get("video_id", ""))
122         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124         rc = int(marker.get("returncode", 1))
125         if not video_id:
126             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                         "video_id) ? treating as failure", self._job_name)
128             rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:

```

```

154         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155     return {}
156

```

31. assistant (2026-06-22T15:04:39.719Z)

This is highly relevant corroborating evidence. Note line 142-155: `\_read\_json` (the sibling read used for `report.json`) ****wraps its fetch+parse in a try/except and returns `{}` on failure****. So the worker side already established the pattern that "a read of a bucket artifact during result assembly must not raise." The `report.json` read at line 130 is graceful; the `intervals.csv` read in `\_ingest\_remote\_segments` is NOT. This is a genuine inconsistency that supports the finding's premise (though `\_read\_json` swallows silently, while the finding asks for a clean `\_failed`, which is arguably better).

Now let me check whether the rc=0 marker guarantees the CSV exists. The worker writes the done-marker with rc=0 ONLY after `\_write\_intervals\_csv` + the push loop succeeds (line 241 ? 248 push ? 260 marker). So in the normal happy path, if rc=0 marker exists, the CSV was pushed to the bucket. The finding's realistic trigger is therefore: GCS eventual-consistency / partial-write / out-of-band deletion / a race ? not the happy path. GCS read-after-write for new objects is strongly consistent, weakening (but not eliminating) the eventual-consistency angle. Let me check the tests added to see what's covered, and whether a missing-CSV case is tested.

32. user (2026-06-22T15:04:41.982Z)

```

5:GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv +
the
38:class _FakeCloudClient(CloudRunJobClient):
39:    """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv + the
done-marker
58:        with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
105:def test_cloud_dispatch_ingests_segments(cloud_env):
122:def test_cloud_dispatch_builds_correct_spec(cloud_env):
139:def test_cloud_failure_marks_failed_no_rows(cloud_env):
153:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
165:def test_default_off_runs_in_process(tmp_path, monkeypatch):
191:def test_full_api_process_to_query_via_cloud(cloud_env):

```

33. assistant (2026-06-22T15:04:47.223Z)

Let me read `test\_cloud\_failure\_marks\_failed\_no\_rows` and the FakeCloud client to confirm there's no missing-CSV test (the finding claims one would lock this in).

34. user (2026-06-22T15:04:48.165Z)

```

38    class _FakeCloudClient(CloudRunJobClient):
39        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
40        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
41
42        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
43            self.calls: list[list[str]] = []
44            self.video_id, self.returncode = video_id, returncode
45            self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
46                                                         (8.74, 11.38, 3, "unforced_error")]
47
48        def run_job(self, job_name, argv, *, region, project=None):
49            self.calls.append(list(argv))
50            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
51            done_uri = argv[argv.index("--done-uri") + 1]
52            if self.returncode == 0:
53                outputs = os.path.join(out_uri, "outputs", self.video_id)
54                os.makedirs(outputs, exist_ok=True)

```



```

55         with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
56             json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
57                       "status": "success"}, f)
58         with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
59             w = csv.writer(f)
60             w.writerow(["start", "end", "shot_count", "ending_reason"])
61             for s, e, sc, er in self.rows:
62                 w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
63             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
64             with open(done_uri, "w", encoding="utf-8") as f:
65                 json.dump({"video_id": self.video_id, "returncode": self.returncode,
66                           "segment_count": len(self.rows), "status": "success"}, f)
67             return "exec-1"
68
69
70     @pytest.fixture
71     def cloud_env(tmp_path, monkeypatch):
72         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
73         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
74         db = Database(str(tmp_path / "t.db"))
75         bucket = tmp_path / "bucket"
76         bucket.mkdir()
77         cfg = AppConfig.model_validate({
78             "deployment": {"compute_target": "cloud_run"},
79             "storage": {"backend": "local", "output_root": str(bucket)},
80         })
81         monkeypatch.setattr(m, "db_instance", db)
82         monkeypatch.setattr(m, "global_config", cfg)
83         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
84         # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
85         fake_local = tmp_path / "fetched.mp4"
86         fake_local.write_bytes(b"FAKEVIDEOBYTES")
87         monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
str(fake_local))
88         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
89         return db, bucket, monkeypatch
90
91
92     def _inject_fake(monkeypatch, fake):
93         """Make backend.compute.get_compute_backend (re-imported inside
_ maybe_dispatch_remote) build a
94         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token."""
95         real = compute_pkg.get_compute_backend
96         monkeypatch.setattr(compute_pkg, "get_compute_backend",
97                             lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))
98
99
100     def _meta(row):
101         md = row.get("metadata")
102         return json.loads(md) if isinstance(md, str) else (md or {})
103
104
105     def test_cloud_dispatch_ingests_segments(cloud_env):
106         db, _bucket, monkeypatch = cloud_env
107         fake = _FakeCloudClient()
108         _inject_fake(monkeypatch, fake)
109
110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",

```

```

segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"

```

35. assistant (2026-06-22T15:05:14.428Z)

I've now gathered all the evidence I need. The finding's test-gap claim is accurate: the `\_FakeCloudClient` always writes `intervals.csv` when `returncode == 0` (line 52-62), so there is no test where rc=0 but the CSV is missing. The `returncode != 0` case (`test\_cloud\_failure\_marks\_failed\_no\_rows`) is the only failure test, and it short-circuits at line 515-516 before the ingest.

Let me do a final assessment by weighing the verification facts against the finding's specific claims.

****What the finding gets RIGHT:****

1. Line citation `main.py:518-520` is exact ? the ingest call site.
2. `\_ingest\_remote\_segments` does `storage.fetch` (line 450) with no error handling, and GCS `download\_to\_filename` raises `NotFound` on a missing blob (gcs.py:38).
3. The call is inside the `try:/finally:` block (no `except`), and the `finally` only emits a report and never raises ? so a raise DOES propagate out of `\_execute\_processing`.
4. This bypasses the `\_failed(...)` shaping that every other cloud error uses.
5. The sibling read `\_read\_json` (cloud_run.py:142-155) wraps fetch+parse in try/except ? establishing that "result-assembly reads shouldn't raise" is already a convention in this very feature. This is strong support that the inconsistency is real.
6. No test covers rc=0-but-missing/garbled-CSV.

****What the finding gets WRONG / overstates:****

1. "becomes a raw 500-shaped internal job error" ? ****incorrect.**** The job worker (`job\_worker.py:96-98`) catches ALL exceptions from `\_execute\_processing` and calls `mark\_job\_failed(job\_id, "NotFound: ...")`. The job lands in a clean terminal `failed` state. It is NOT a raw 500 (the API call already returned 202 at submit time; this runs on a worker thread). So the actual blast radius is much smaller than "raw 500-shaped internal job error."
2. The "empty-rally run that simply didn't emit the CSV" sub-scenario is ****incorrect**** ? the worker unconditionally writes `intervals.csv` (header-only) even for zero segments (worker/`\_\_main\_\_.py:241`), and rc=0 marker is written only AFTER the push. So the happy path always has the CSV.
3. The two material differences vs `\_failed` are: (a) less specific error message ("NotFound: ..." vs "could not read intervals.csv from <uri>"), and (b) no `prune\_segments` call. But (b) is harmless here because `\_ingest\_remote\_segments` fetches BEFORE writing any rows ? if `fetch` raises, zero new rows exist, so there is nothing to prune and prior good rows are untouched. So the data-safety claim is also moot for the fetch-failure case.

****Net severity assessment:**** The defect is real but mischaracterized in severity. The actual consequence is: a missing/garbled CSV after rc=0 yields a `mark\_job\_failed` with a generic `"NotFound: 404 ..."` message instead of a clean, self-describing `\_failed("cloud-serving: could not read intervals.csv ...")`. That is a robustness/observability inconsistency, not a crash that escapes to a 500 or corrupts data. The triggering conditions (partial write / out-of-band deletion / race) are plausible but uncommon given GCS strong read-after-write consistency and the marker-written-last ordering. The fix is cheap and correct, and the inconsistency with the sibling `\_read\_json` is a legitimate code-quality point.

This is a real but ****minor**** issue ? the worker's catch-all already converts it to a clean terminal `failed` job state with the exception text; the only loss is a precise error message and parity with the surrounding `\_failed`/`\_read\_json` patterns. It does not crash the service, does not produce a 500, and does not corrupt or orphan DB rows.

36. user (2026-06-22T15:05:23.340Z)

Structured output provided successfully

37. assistant (2026-06-22T15:05:31.903Z)

I've verified the finding against the actual code in C:/Users/avidu/Projects/bhi-cloudrun.

Verdict: real defect, but minor ? not major as claimed.

The mechanism holds up exactly as cited: ``_ingest_remote_segments`` (backend/main.py:441-468) does an unguarded ``storage.fetch`` at line 450, GCS raises ``NotFound`` on a missing blob (gcs.py:38), and the call site at main.py:520 sits inside a ``try:/finally:`` with no ``except``, so the exception escapes ``_execute_processing`` and bypasses the ``_failed(...)`` shaping used by every other cloud branch. The sibling read ``_read_json`` (cloud_run.py:142-155) already wraps `fetch+parse` in `try/except`, confirming the inconsistency is real, and no test covers the `rc=0-but-missing-CSV` case.

But the severity is overstated on three counts:

1. It does NOT become a "raw 500-shaped internal job error" ? the job worker catches all exceptions (job_worker.py:96-98) and calls ``mark_job_failed``, so the job lands in a clean terminal ``failed`` state with the exception text. The HTTP API already returned 202 at submit.
2. The "empty-rally run that didn't emit the CSV" trigger is wrong ? the worker writes ``intervals.csv`` unconditionally (worker/___main__.py:241), header-only for zero rallies, and the `rc=0` marker is written afterward.
3. The data-safety concern (missing prune) is moot ? ``fetch`` runs before any row write, so a fetch failure writes nothing and leaves prior rows intact.

Net: the only real loss versus ``_failed`` is a precise, self-describing error message and parity with the surrounding patterns, under uncommon triggers (partial write, out-of-band deletion, race; GCS new-object reads are strongly consistent). The suggested fix is correct and cheap, worth doing as robustness/observability polish, but it is not a blocker or a crash/data-corruption bug.